

Programação Orientada a Objetos

Eduardo Kessler Piveta

Universidade Federal de Santa Maria - UFSM

Programação Estruturada

- A programação estruturada fornece mecanismos para modularizar os conceitos envolvidos em um sistema de software através da definição de estruturas de dados e dos algoritmos que as manipulam.
- As linguagens estruturadas possibilitam a representação de diferentes tipos de dados e de **variáveis** de forma a expressar as características e propriedades dos conceitos envolvidos, bem como **procedimentos e funções** para modelar as operações envolvidas.

Programação Estruturada - Problemas

- Ao tentarmos reusar componentes de linguagens estruturadas, precisamos descobrir a relação entre os dados (representados pelas estruturas de dados, tipos de dados e variáveis) e o comportamento (expresso através de procedimentos e funções).
- Desta forma, o reuso normalmente ocorre em uma granularidade fina (uma estrutura, um método) e muitas vezes demanda um esforço considerável (para identificar, por exemplo, quais procedimentos e funções relacionam-se com quais estruturas de dados).
- Embora seja possível a definição de bibliotecas reutilizáveis, a relação entre dados e comportamento não é explícita e localizada em uma única abstração.

Programação Orientada a Objetos

- A **programação orientada a objetos** visa representar os conceitos de um determinado domínio de aplicação através da criação de tipos abstratos de dados, chamados classes, que representem tanto as características e propriedades desses conceitos quanto suas operações.
- A POO fornece também mecanismos de composição entre esses conceitos, melhorando o reuso e a modularidade dos componentes de um sistema.
- É o paradigma mais utilizado no desenvolvimento de sistemas de software. Diversas linguagens popularmente usadas fornecem suporte a este paradigma, incluindo: Java, C++, C#, Python e Ruby.

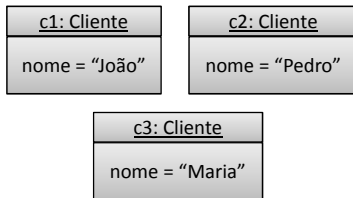
Classes

- Uma **classe** é um mecanismo de abstração usado para representar conceitos de um determinado domínio de aplicação.
- Por exemplo, se considerarmos um domínio bancário, os conceitos seriam relacionados a agências, contas, clientes, transações, etc.



Objetos

- As classes servem como *templates* para a criação de exemplares concretos e manipuláveis desses conceitos, chamados de **objetos**.
 - Diz-se que um objeto é uma **instância** de uma classe.
 - O processo de criação de objetos é chamado de **instanciação**.
- Exemplo: objetos da classe Cliente.



Atributos e Métodos

Uma classe é definida de forma a representar as propriedades, as características e o comportamento de um conceito. Para isso, são utilizados principalmente dois mecanismos:

- **Atributos:** são utilizados para representar características ou propriedades dos objetos de uma classe.
- **Métodos:** são utilizados para representar o comportamento dos objetos.

Atributos

- Exemplos de atributos de uma classe **Cliente** seriam *nome*, *idade*, *endereço* e *estado civil*.
- Dessa forma, ao ser instanciado, cada cliente específico poderia possuir valores distintos para esses atributos.
- Por exemplo, um objeto representando um cliente poderia ter os seguintes valores de atributos:
 - *nome* = João da Silva
 - *idade* = 50
 - *endereço* = Rua Presidente Vargas, 205
 - *estado civil* = casado

Métodos

- Por exemplo, possíveis métodos de uma classe *Conta* poderiam estar relacionados às operações de *abrir*, *encerrar*, ou *alterar o titular* de uma conta.
- A **invocação** de um método (também conhecida como **chamada** a um método) de uma classe faz com que a operação implementada no método seja executada.
- Comumente, as classes possuem métodos especiais para a criação (instanciação) de objetos, chamados de **construtores** e para a destruição de objetos, chamados de **destrutores**.

Membros de instância e de classe

- Os **atributos e métodos de instância** são os mais comumente utilizados e são acessados a partir de objetos da classe. Eles representam características e operações que estão relacionadas com as instâncias individualmente.
 - Exemplos de atributos de instância de uma classe Conta seriam: *banco, agência, número da conta, titular da conta*, etc.
 - Métodos de instância seriam operações para uma conta específica, tais como: *sacar, transferir, depositar*, etc.
- Já os **atributos e métodos de classe** representam atributos e métodos da classe em si (e não de suas instâncias) como, por exemplo, um atributo que conta o número de instâncias criadas pela classe Conta ou um método que obtém tal número de instâncias desse atributo.

Encapsulamento e Ocultação de Informação

- De forma a restringir o acesso a atributos e métodos, as classes fornecem mecanismos de **encapsulamento e ocultação de informação**.
- Normalmente o que as linguagens de programação fornecem é o uso de modificadores de visibilidade, que definem quem consegue enxergar um determinado atributo, método, ou classe.

Relacionamentos

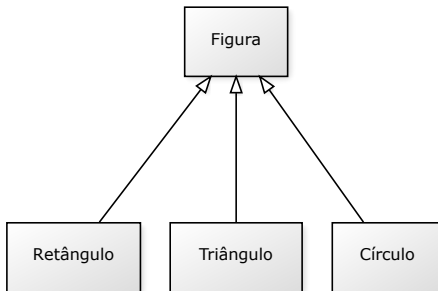
- As classes servem como mecanismos de abstração, nas quais conceitos do mundo real e do mundo computacional são representados e encapsulados em uma estrutura de classe.
- A partir dessas abstrações, são utilizados mecanismos de composição (relacionamentos) de forma a unir tais classes para que elas possam auxiliar a atender os requisitos da aplicação. Dentre os diferentes relacionamentos possíveis entre classes, destacam-se: herança, associação, agregação, composição e realização.

Herança

- A **herança** é uma mecanismo de composição que permite que as classes reusem a interface e o comportamento de classes existentes, através da extensão de suas funcionalidades de maneira aditiva.
- Considere uma classe A e uma classe B que herda da (ou estende a) classe A. Diz-se que a classe A é uma superclasse de B e que B é uma subclasse de A.
- O usual é que a definição de extensão esteja especificada na subclasse (algo como: SubClasse estende SuperClasse).

Herança - Exemplo

- Considere, por exemplo, as classes Figura, Círculo, Triângulo e Retângulo. Nesse caso, a superclasse é a classe Figura e as subclasses são Círculo, Triângulo e Retângulo.



Herança

- O mecanismo de herança pode variar de acordo com a linguagem de programação e tais variabilidades normalmente dizem respeito à visibilidade e à multiplicidade da herança.
- Algumas linguagens, como C++, permitem que a **visibilidade da herança** seja especificada como **pública** ou como **privada**.
- E algumas linguagens, como C++, permitem que a herança seja múltipla (uma classe tenha mais de uma superclasse).

Classes abstratas

- Algumas classes são ditas **abstratas**. Tais classes representam as características e o comportamento de um conjunto de objetos do mundo real da mesma forma que as demais classes (ditas **concretas**).
- No entanto, não existem instâncias diretas de tais classes (elas não podem ser instanciadas).
- Por exemplo, no caso da herança múltipla, poderíamos definir as classes Mamífero, Ave e Voador como classes abstratas, pois não há exemplares diretos dessas classes no mundo real - há apenas exemplares de suas subclasses (cavalos, cachorros, patos, etc).

Polimorfismo

- O **polimorfismo** é a capacidade que uma operação tem de ter um comportamento diferente dependendo da classe do objeto que a chama. As principais formas de polimorfismo são:
 - **Polimorfismo ad-hoc (ou de sobrecarga)**, que ocorre quando existem métodos de mesmo nome com parâmetros formais com tipos diferentes entre si.
 - **Polimorfismo paramétrico**, que ocorre quando um método é criado de maneira genérica, vinculado a um tipo que só é definido em tempo de instanciação.
 - **Polimorfismo de subtipo**, que ocorre quando as subclasses sobrescrevem métodos de uma de suas superclasses. Nesse caso, a determinação de qual método será executado dependerá do objeto que está fazendo a chamada.

Interfaces

- Uma **interface** pode ser vista como uma classe abstrata com todos os métodos abstratos e serve como um protocolo entre os clientes e os implementadores da interface.
- Nesse caso, a classe (ou uma de suas superclasses) deve implementar os métodos descritos na interface.
- Dessa forma, os clientes podem implementar código que satisfaça a interface sem se preocupar em como as classes que a implementam o fazem.
- A **realização**, ocorre quando uma classe define que implementa uma determinada interface.

Conclusões

- A orientação a objetos fornece mecanismos de abstração e de composição para representar as principais abstrações do domínio de aplicação que se deseja e uní-las de forma a modelar um sistema como um todo.
- As classes podem ser compostas através de herança e relacionamentos (agregação, associação, etc), bem como através do uso de interfaces.
- O uso de herança possibilita o aproveitamento do polimorfismo de diferentes maneiras, facilitando o reuso de código.